UNIVERSIDAD CATÓLICA DE SANTA MARÍA
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS
LENGUAJES DE PROGRAMACIÓN III

PRÁCTICA N.° 08: BASES DE DATOS SQLITE Y JDBC EN JAVA

Código	Apellidos y Nombres	Fecha
2025002762	Rodríguez Fádel, Gian Piero Khalil	21/10/2026

Caso D: Plataforma de Juego Multijugador.

1. ACTIVIDADES

La experiencia de práctica se desarrolló a partir del caso D asignado: una plataforma de juego multijugador. Se construyó el modelo relacional y una aplicación Java en Eclipse que crea y manipula SQLite mediante JDBC. La actividad del apartado IV pide diseñar el modelo y aplicar operaciones de inserción, recuperación, actualización, eliminación, consultas parametrizadas y transacciones sobre una tabla elegida (UCSM, 2026).

1.1 Análisis del caso de estudio

Los usuarios se registran en la plataforma, pueden configurar su perfil, participar en partidas de diversos juegos, acumular estadísticas y desbloquear logros. Una partida pertenece a un juego; un usuario puede registrar participaciones en muchas partidas a lo largo del tiempo. Las participaciones se conservan como historial, y desde Java se verifica que un jugador no ingrese simultáneamente a otra partida marcada EN_CURSO.

1.2 Diseño del modelo entidad-relación

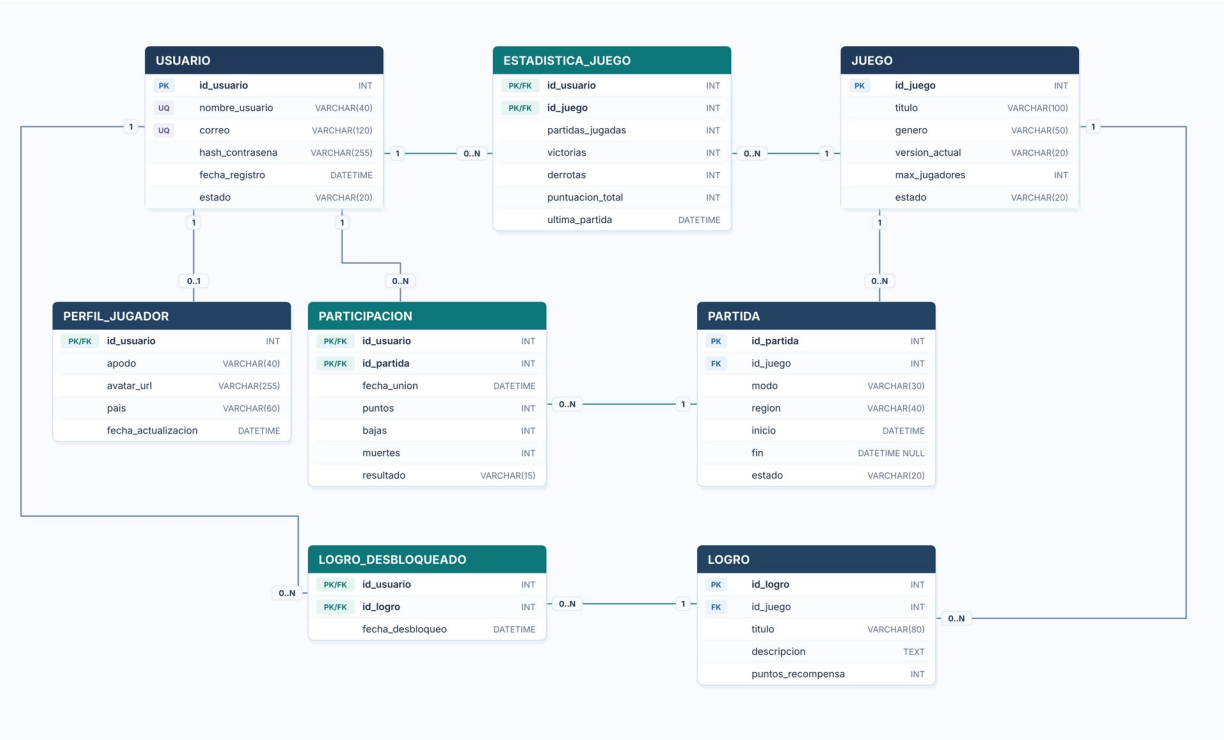


Figura 1. Modelo entidad-relación con ocho tablas y relaciones 1:1, 1:N y N:M.

Entidad	Clave primaria	Relación principal
usuario	id_usuario	1:1 con perfil_jugador
perfil_jugador	id_usuario	PK y FK hacia usuario
juego	id_juego	1:N con partida y logro
partida	id_partida	FK hacia juego
participacion	id_usuario, id_partida	Intermedia: usuario ↔ partida
estadistica_juego	id_usuario, id_juego	Intermedia: usuario ↔ juego
logro	id_logro	FK hacia juego
logro_desbloqueado	id_usuario, id_logro	Intermedia: usuario ↔ logro

El modelo supera el mínimo de cuatro tablas de la actividad y el mínimo de seis de los ejercicios. Las relaciones compuestas impiden duplicar una misma participación o desbloqueo. La validación referencial se habilita por conexión con PRAGMA foreign_keys = ON, como indica la documentación de SQLite (SQLite, s. f.-a).

1.3 Configuración de Eclipse y conexión JDBC

El proyecto ejemplo utiliza el módulo java.sql del JDK y el controlador externo sqlite-jdbc. Se conserva la ruta al archivo JAR que figuraba en el proyecto original; al importar el proyecto en otro equipo debe corregirse esa ruta local en las propiedades de Eclipse. La conexión jdbc:sqlite:multijugador.db abre o crea el archivo de datos según el directorio de ejecución. La clase BaseDeDatos.java centraliza la conexión y crea las ocho tablas solo cuando faltan. Su código completo y evidencia se encuentran en el apartado 2.2.1.

1.4 Operaciones solicitadas en la actividad

Se seleccionó la entidad usuario para demostrar las operaciones CRUD. Con PreparedStatement, los datos se suministran a parámetros ? en lugar de concatenar directamente valores en el texto SQL. Se utiliza ResultSet para recorrer el resultado de consultas; para cambios se ejecuta executeUpdate(). Cada operación que modifica datos solicita una clave de confirmación y después realiza commit() o rollback() de la transacción, según corresponda (Oracle, s. f.-a; Oracle, s. f.-b).

Operación de actividad	Implementación / comprobación
Insertar registros	GestorCRUD.insertar() y DatosPrueba.cargar()
Recuperar registros	GestorCRUD.mostrar() y Consultas.ejecutar()
Actualizar registros	GestorCRUD.actualizar() y EjerciciosSQL.aplicarCambios()
Eliminar registros	GestorCRUD.eliminar() y EjerciciosSQL.aplicarCambios()
PreparedStatement	DatosPrueba, Consultas, EjerciciosSQL y GestorCRUD
Transacciones	Principal, GestorCRUD y Seguridad: commit / rollback

1.5 Resultado de la actividad

Se implementó la estructura relacional y se incluyeron las operaciones requeridas. La verificación ejecutada directamente en SQLite obtuvo ocho tablas, diez registros iniciales en cada una, siete consultas funcionales y pruebas satisfactorias de restricciones y transacciones. La compilación del código fuente con Java 21 también se comprobó. La ejecución del controlador sqlite-jdbc dentro de Eclipse queda como comprobación del entorno del estudiante; las imágenes de consola de este informe se identifican expresamente como simulaciones.

2. EJERCICIOS PROPUESTOS

Los seis ejercicios del apartado V se desarrollan a continuación. Todo el código fuente correspondiente al proyecto se presenta íntegro dentro de esta sección, acompañado por la explicación de cada archivo y una simulación visual de su ejecución. No se incluyen bloques de código en la sección Actividades.

2.1 Ejercicios 1 y 2: seis tablas y modelo entidad-relación

Los ejercicios solicitan crear una base de datos con al menos seis tablas, incluir relaciones 1:1, 1:N y N:M, y elaborar previamente el modelo ER. Se diseñaron ocho entidades y se utiliza el esquema mostrado en la Figura 1. USUARIO y PERFIL_JUGADOR forman la relación 1:1; JUEGO y PARTIDA representan 1:N; PARTICIPACION, ESTADISTICA_JUEGO y LOGRO_DESBLOQUEADO implementan relaciones N:M mediante claves compuestas.

2.2 Ejercicio 3: creación, datos y consultas SQL

La base se crea desde Java y conserva FOREIGN KEY en las dependencias. Se cargan al menos diez registros por tabla con PreparedStatement; a continuación se ejecutan siete SELECT con condiciones, tres UPDATE y tres DELETE con WHERE. En las pruebas se mantienen activas las claves foráneas y se confirma o revierte el conjunto de operaciones con una sola transacción.

2.2.1 Creación de las ocho tablas

BaseDeDatos mantiene el código que ya existía en el repositorio, sin cambios. La llamada a inicializar() ejecuta CREATE TABLE IF NOT EXISTS sobre todas las entidades. getConnection() configura PRAGMA foreign_keys = ON antes de utilizar la conexión. Esta inicialización es repetible, pero no migra estructuras existentes (SQLite, s. f.-b).

Archivo: ejemplo/src/conexion/BaseDeDatos.java

```
package conexion;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class BaseDeDatos {
    private static final String URL = "jdbc:sqlite:multijugador.db";

    public static Connection getConnection() throws SQLException
    {
        Connection con = DriverManager.getConnection(URL);

        try
        {
            try(Statement stmt = con.createStatement())
            {
                stmt.execute("PRAGMA foreign_keys = ON");
            }
            return con;
        }
        catch(SQLException e)
        {
            con.close();
            throw e;
        }
    }

    public static void inicializar() throws SQLException
    {
        try(Connection con = getConnection(); Statement stmt = con.createStatement())
        {
            String sqlUsuario = """
                    CREATE TABLE IF NOT EXISTS usuario
                    (
                        id_usuario INTEGER PRIMARY KEY,
                        nombre_usuario TEXT NOT NULL UNIQUE,
                        correo TEXT NOT NULL UNIQUE,
                        hash_contrasena TEXT NOT NULL,
```

```
            fecha_registro TEXT DEFAULT CURRENT_TIMESTAMP,
            estado TEXT NOT NULL
        )
        """;
stmt.execute(sqlUsuario);

String sqlPerfil = """
        CREATE TABLE IF NOT EXISTS perfil_jugador
        (
            id_usuario INTEGER PRIMARY KEY,
            apodo TEXT,
            avatar_url TEXT,
            pais TEXT,
            fecha_actualizacion DEFAULT CURRENT_TIMESTAMP,

            FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)
        )
        """;
stmt.execute(sqlPerfil);

String sqlJuego = """
        CREATE TABLE IF NOT EXISTS juego
        (
            id_juego INTEGER PRIMARY KEY,
            titulo TEXT NOT NULL,
            genero TEXT,
            version_actual TEXT NOT NULL,
            max_jugadores INTEGER NOT NULL,
            estado TEXT NOT NULL
        )
        """;
stmt.execute(sqlJuego);

String sqlPartida = """
        CREATE TABLE IF NOT EXISTS partida
        (
            id_partida INTEGER PRIMARY KEY,
            id_juego INTEGER NOT NULL,
            modo TEXT NOT NULL,
            region TEXT NOT NULL,
            inicio TEXT DEFAULT CURRENT_TIMESTAMP NOT NULL,
            fin TEXT,
            estado TEXT NOT NULL,

            FOREIGN KEY (id_juego) REFERENCES juego(id_juego)
        )
        """;
stmt.execute(sqlPartida);

String sqlParticipacion = """
        CREATE TABLE IF NOT EXISTS participacion
        (
            id_usuario INTEGER NOT NULL,
            id_partida INTEGER NOT NULL,
            fecha_union TEXT DEFAULT CURRENT_TIMESTAMP,
            puntos INTEGER NOT NULL DEFAULT 0,
            bajas INTEGER NOT NULL DEFAULT 0,
            muertes INTEGER NOT NULL DEFAULT 0,
            resultado TEXT,

            PRIMARY KEY(id_usuario, id_partida),

            FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario),
            FOREIGN KEY (id_partida) REFERENCES partida(id_partida)
        )
        """;
stmt.execute(sqlParticipacion);

String sqlEstadistica = """
        CREATE TABLE IF NOT EXISTS estadistica_juego
        (
            id_usuario INTEGER NOT NULL,
            id_juego INTEGER NOT NULL,
            partidas_jugadas INTEGER NOT NULL DEFAULT 0,
            victorias INTEGER NOT NULL DEFAULT 0,
            derrotas INTEGER NOT NULL DEFAULT 0,
            puntuacion_total INTEGER NOT NULL DEFAULT 0,
            ultima_partida TEXT,

            PRIMARY KEY (id_usuario, id_juego),

            FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario),
```

```
                    FOREIGN KEY (id_juego) REFERENCES juego(id_juego)
                )
                """;
            stmt.execute(sqlEstadistica);

            String sqlLogro = """
                    CREATE TABLE IF NOT EXISTS logro
                    (
                        id_logro INTEGER PRIMARY KEY,
                        id_juego INTEGER NOT NULL,
                        titulo TEXT NOT NULL,
                        descripcion TEXT,
                        puntos_recompensa INTEGER NOT NULL DEFAULT 0,

                        FOREIGN KEY (id_juego) REFERENCES juego(id_juego)
                    )
                    """;
            stmt.execute(sqlLogro);

            String sqlLogroDesbloqueado = """
                    CREATE TABLE IF NOT EXISTS logro_desbloqueado
                    (
                        id_usuario INTEGER NOT NULL,
                        id_logro INTEGER NOT NULL,
                        fecha_desbloqueo TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,

                        PRIMARY KEY (id_usuario, id_logro),

                        FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario),
                        FOREIGN KEY (id_logro) REFERENCES logro(id_logro)
                    )
                    """;
            stmt.execute(sqlLogroDesbloqueado);
        }
    }
}
```

Explicación del archivo: Centraliza DriverManager.getConnection() con la URL SQLite, activa las claves foráneas para cada conexión y ejecuta las ocho instrucciones DDL. Al estar creada la tabla, IF NOT EXISTS no altera su definición. Se conservó exactamente el archivo de partida.

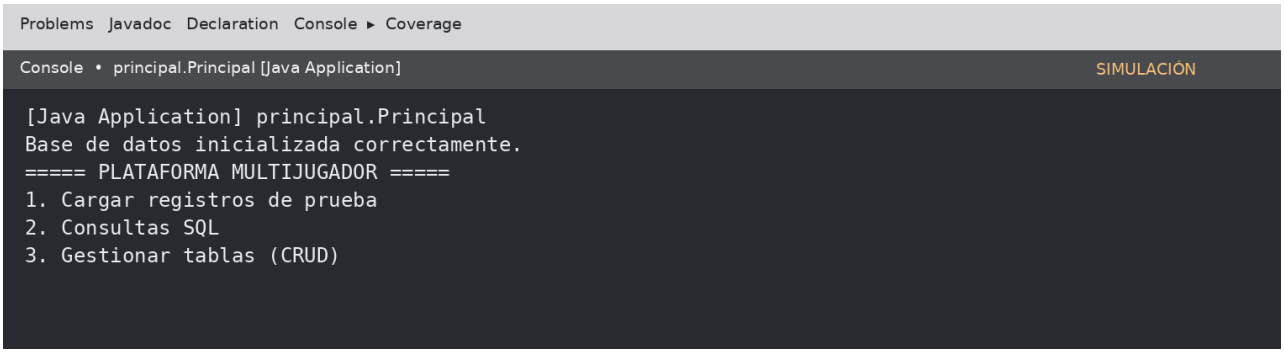


Figura 2. Simulación de la inicialización de SQLite desde la consola de Eclipse.

2.2.2 Carga mínima de diez registros por tabla

Los datos de prueba respetan el orden de dependencias: primero usuario y juego; luego perfil, partidas, participaciones, estadísticas y logros. Se usan claves fijas para comprobar relaciones y INSERT OR IGNORE para evitar duplicados cuando se repite una carga. Los valores HASH_DE_PRUEBA son marcadores didácticos: no son contraseñas cifradas ni deben usarse como credenciales reales.

Archivo: ejemplo/src/conexion/DatosPrueba.java

```
package conexion;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class DatosPrueba {
    public static void cargar(Connection con) throws SQLException
    {
        insertar(con, "INSERT OR IGNORE INTO usuario (id_usuario, nombre_usuario, correo, hash_contrasena, estado)
VALUES (?, ?, ?, ?, ?)", new Object[][] {
```

```
                {1, "Piero", "piero@ejemplo.com", "HASH_DE_PRUEBA_1", "ACTIVO"},
                {2, "Andrea", "andrea@ejemplo.com", "HASH_DE_PRUEBA_2", "ACTIVO"},
                {3, "Carlos", "carlos@ejemplo.com", "HASH_DE_PRUEBA_3", "ACTIVO"},
                {4, "Lucia", "lucia@ejemplo.com", "HASH_DE_PRUEBA_4", "ACTIVO"},
                {5, "Diego", "diego@ejemplo.com", "HASH_DE_PRUEBA_5", "ACTIVO"},
                {6, "Camila", "camila@ejemplo.com", "HASH_DE_PRUEBA_6", "ACTIVO"},
                {7, "Mateo", "mateo@ejemplo.com", "HASH_DE_PRUEBA_7", "ACTIVO"},
                {8, "Valeria", "valeria@ejemplo.com", "HASH_DE_PRUEBA_8", "ACTIVO"},
                {9, "Sofia", "sofia@ejemplo.com", "HASH_DE_PRUEBA_9", "ACTIVO"},
                {10, "Bruno", "bruno@ejemplo.com", "HASH_DE_PRUEBA_10", "ACTIVO"}
        });

        insertar(con, "INSERT OR IGNORE INTO perfil_jugador (id_usuario, apodo, avatar_url, pais) VALUES (?, ?, ?,
?)", new Object[][] {
                {1, "PieroX", "avatars/1.png", "Peru"}, {2, "Andromeda", "avatars/2.png", "Peru"},
                {3, "CarlosGG", "avatars/3.png", "Chile"}, {4, "Luci4", "avatars/4.png", "Peru"},
                {5, "D_Rex", "avatars/5.png", "Mexico"}, {6, "Camix", "avatars/6.png", "Peru"},
                {7, "Mateo7", "avatars/7.png", "Argentina"}, {8, "Valkyrie", "avatars/8.png", "Peru"},
                {9, "SofiStar", "avatars/9.png", "Chile"}, {10, "BRun", "avatars/10.png", "Colombia"}
        });

        insertar(con, "INSERT OR IGNORE INTO juego (id_juego, titulo, genero, version_actual, max_jugadores,
estado) VALUES (?, ?, ?, ?, ?, ?)", new Object[][] {
                {1, "Arena Nova", "Accion", "1.2", 10, "ACTIVO"},
                {2, "Reino Pixel", "RPG", "2.0", 8, "ACTIVO"},
                {3, "Velocidad X", "Carreras", "1.5", 12, "ACTIVO"},
                {4, "Titan Clash", "Estrategia", "3.1", 6, "ACTIVO"},
                {5, "Galaxia Sur", "Aventura", "1.7", 4, "ACTIVO"},
                {6, "Guardianes", "Accion", "1.1", 10, "ACTIVO"},
                {7, "Cubo Magico", "Puzzle", "2.2", 4, "ACTIVO"},
                {8, "Mundo Mar", "Supervivencia", "1.3", 6, "ACTIVO"},
                {9, "Zona Roja", "Accion", "1.0", 12, "ACTIVO"},
                {10, "Fortaleza", "Estrategia", "2.5", 8, "ACTIVO"}
        });

        insertar(con, "INSERT OR IGNORE INTO partida (id_partida, id_juego, modo, region, inicio, fin, estado)
VALUES (?, ?, ?, ?, ?, ?, ?)", new Object[][] {
                {101, 1, "Equipos", "LATAM", "2026-10-18 18:00:00", "2026-10-18 18:35:00", "FINALIZADA"},
                {102, 1, "Equipos", "LATAM", "2026-10-19 19:00:00", "2026-10-19 19:40:00", "FINALIZADA"},
                {103, 2, "Cooperativo", "LATAM", "2026-10-19 20:00:00", "2026-10-19 20:30:00", "FINALIZADA"},
                {104, 2, "Duelo", "EU", "2026-10-19 21:00:00", "2026-10-19 21:20:00", "FINALIZADA"},
                {105, 3, "Carrera", "LATAM", "2026-10-20 16:00:00", "2026-10-20 16:15:00", "FINALIZADA"},
                {106, 3, "Carrera", "NA", "2026-10-20 16:20:00", "2026-10-20 16:40:00", "FINALIZADA"},
                {107, 4, "Equipos", "LATAM", "2026-10-20 17:00:00", "2026-10-20 17:25:00", "FINALIZADA"},
                {108, 4, "Duelo", "EU", "2026-10-20 18:00:00", "2026-10-20 18:20:00", "FINALIZADA"},
                {109, 5, "Cooperativo", "LATAM", "2026-10-21 21:00:00", null, "EN_CURSO"},
                {110, 5, "Cooperativo", "LATAM", "2026-10-21 21:30:00", null, "ESPERA"}
        });

        insertar(con, "INSERT OR IGNORE INTO participacion (id_usuario, id_partida, puntos, bajas, muertes,
resultado) VALUES (?, ?, ?, ?, ?, ?)", new Object[][] {
                {1, 101, 170, 14, 6, "VICTORIA"}, {2, 101, 90, 8, 9, "DERROTA"},
                {1, 102, 120, 11, 8, "VICTORIA"}, {3, 102, 80, 7, 10, "DERROTA"},
                {4, 103, 150, 12, 3, "VICTORIA"}, {5, 104, 50, 3, 12, "DERROTA"},
                {6, 105, 200, 0, 0, "VICTORIA"}, {7, 106, 70, 0, 0, "DERROTA"},
                {8, 107, 140, 8, 5, "VICTORIA"}, {9, 109, 0, 0, 0, null}
        });

        insertar(con, "INSERT OR IGNORE INTO estadistica_juego (id_usuario, id_juego, partidas_jugadas, victorias,
derrotas, puntuacion_total, ultima_partida) VALUES (?, ?, ?, ?, ?, ?, ?)", new Object[][] {
                {1, 1, 2, 2, 0, 290, "2026-10-19 19:40:00"},
                {2, 1, 1, 0, 1, 90, "2026-10-18 18:35:00"},
                {3, 1, 1, 0, 1, 80, "2026-10-19 19:40:00"},
                {4, 2, 1, 1, 0, 150, "2026-10-19 20:30:00"},
                {5, 2, 1, 0, 1, 50, "2026-10-19 21:20:00"},
                {6, 3, 1, 1, 0, 200, "2026-10-20 16:15:00"},
                {7, 3, 1, 0, 1, 70, "2026-10-20 16:40:00"},
                {8, 4, 1, 1, 0, 140, "2026-10-20 17:25:00"},
                {9, 5, 0, 0, 0, 0, null},
                {10, 5, 0, 0, 0, 0, null}
        });

        insertar(con, "INSERT OR IGNORE INTO logro (id_logro, id_juego, titulo, descripcion, puntos_recompensa)
VALUES (?, ?, ?, ?, ?)", new Object[][] {
                {1, 1, "Primera partida", "Disputa una partida", 100},
                {2, 1, "Dos victorias", "Gana dos partidas", 200},
                {3, 2, "Explorador", "Descubre un area", 50},
                {4, 2, "Maestro", "Supera una mision", 150},
                {5, 3, "Rapido", "Gana una carrera", 100},
                {6, 3, "Piloto", "Participa en una carrera", 300},
                {7, 4, "Estratega", "Gana un duelo", 100},
                {8, 4, "Comandante", "Gana cinco duelos", 250},
```

```
                {9, 5, "Pionero", "Inicia una aventura", 50},
                {10, 5, "Leyenda", "Completa la aventura", 400}
        });

        insertar(con, "INSERT OR IGNORE INTO logro_desbloqueado (id_usuario, id_logro, fecha_desbloqueo) VALUES
(?, ?, ?)", new Object[][] {
                {1, 1, "2026-10-18 18:35:00"}, {1, 2, "2026-10-19 19:40:00"},
                {2, 1, "2026-10-18 18:35:00"}, {3, 1, "2026-10-19 19:40:00"},
                {4, 3, "2026-10-19 20:30:00"}, {5, 4, "2026-10-19 21:20:00"},
                {6, 5, "2026-10-20 16:15:00"}, {7, 6, "2026-10-20 16:40:00"},
                {8, 7, "2026-10-20 17:25:00"}, {9, 9, "2026-10-21 21:00:00"}
        });
    }

    private static void insertar(Connection con, String sql, Object[][] registros) throws SQLException
    {
        try(PreparedStatement ps = con.prepareStatement(sql))
        {
            for(Object[] registro : registros)
            {
                for(int i = 0; i < registro.length; i++)
                {
                    ps.setObject(i + 1, registro[i]);
                }
                ps.executeUpdate();
            }
        }
    }
}
```

Explicación del archivo: Agrupa diez registros de prueba de cada entidad. El método insertar prepara una sentencia por conjunto de datos, asigna los parámetros mediante setObject() y ejecuta la inserción. Los datos conservan la integridad referencial.

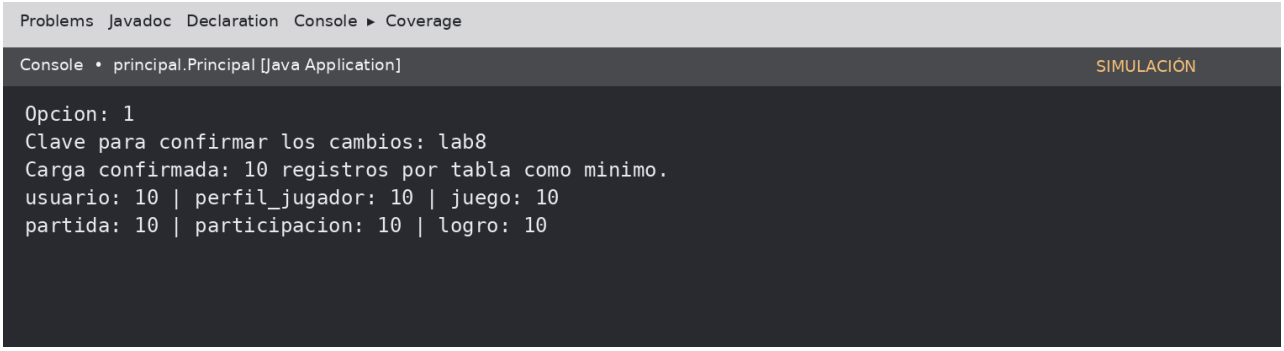


Figura 3. Simulación de la inserción inicial mediante PreparedStatement.

2.2.3 Siete consultas SQL con condicionales

N.º	Consulta	Condición y relación
1	Usuarios activos	usuario.estado = ACTIVO
2	Juegos según cupo	juego.max_jugadores >= valor
3	Partidas de un juego	partida.id_juego = valor, JOIN juego
4	Historial de usuario	participacion.id_usuario = valor, JOIN partida
5	Participaciones por puntos	participacion.puntos >= mínimo
6	Partidas terminadas por región	partida.estado = FINALIZADA y región
7	Logros desbloqueados	logro_desbloqueado.id_usuario = valor, JOIN logro

Archivo: ejemplo/src/consultas/Consultas.java

```
package consultas;

import java.sql.Connection;
```

```
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.ResultSetMetaData;
import java.sql.SQLException;
import java.util.Scanner;

import conexion.BaseDeDatos;
import principal.Principal;

public class Consultas {
    public static void menu(Scanner scanner) throws SQLException
    {
        int opcion;
        do
        {
            System.out.println("\n===== CONSULTAS CON CONDICIONES =====");
            System.out.println("1. Usuarios activos");
            System.out.println("2. Juegos por capacidad minima");
            System.out.println("3. Partidas de un juego");
            System.out.println("4. Historial de un usuario");
            System.out.println("5. Jugadores con puntaje minimo");
            System.out.println("6. Partidas finalizadas por region");
            System.out.println("7. Logros de un usuario");
            System.out.println("0. Volver");
            System.out.print("Opcion: ");
            opcion = Principal.leerNumero(scanner);
            switch(opcion)
            {
                case 1 -> ejecutar("SELECT id_usuario, nombre_usuario, estado FROM usuario WHERE estado = ? ORDER
BY nombre_usuario", "ACTIVO");
                case 2 -> {
                    System.out.print("Capacidad minima: ");
                    ejecutar("SELECT id_juego, titulo, max_jugadores FROM juego WHERE max_jugadores >= ? ORDER BY
max_jugadores DESC", Principal.leerNumero(scanner));
                }
                case 3 -> {
                    System.out.print("ID del juego: ");
                    ejecutar("SELECT p.id_partida, j.titulo, p.modo, p.estado FROM partida p JOIN juego j ON
p.id_juego = j.id_juego WHERE j.id_juego = ? ORDER BY p.id_partida", Principal.leerNumero(scanner));
                }
                case 4 -> {
                    System.out.print("ID del usuario: ");
                    ejecutar("SELECT u.nombre_usuario, j.titulo, p.id_partida, pa.puntos, pa.resultado FROM
participacion pa JOIN usuario u ON pa.id_usuario = u.id_usuario JOIN partida p ON pa.id_partida = p.id_partida
JOIN juego j ON p.id_juego = j.id_juego WHERE u.id_usuario = ? ORDER BY p.inicio DESC",
Principal.leerNumero(scanner));
                }
                case 5 -> {
                    System.out.print("Puntaje minimo: ");
                    ejecutar("SELECT u.nombre_usuario, pa.id_partida, pa.puntos, pa.bajas FROM participacion pa
JOIN usuario u ON pa.id_usuario = u.id_usuario WHERE pa.puntos >= ? ORDER BY pa.puntos DESC",
Principal.leerNumero(scanner));
                }
                case 6 -> {
                    System.out.print("Region (LATAM, EU o NA): ");
                    ejecutar("SELECT p.id_partida, j.titulo, p.region, p.fin FROM partida p JOIN juego j ON
p.id_juego = j.id_juego WHERE p.estado = ? AND p.region = ? ORDER BY p.fin DESC", "FINALIZADA",
scanner.nextLine().trim());
                }
                case 7 -> {
                    System.out.print("ID del usuario: ");
                    ejecutar("SELECT u.nombre_usuario, l.titulo, j.titulo AS juego, ld.fecha_desbloqueo FROM
logro_desbloqueado ld JOIN usuario u ON ld.id_usuario = u.id_usuario JOIN logro l ON ld.id_logro = l.id_logro JOIN
juego j ON l.id_juego = j.id_juego WHERE u.id_usuario = ? ORDER BY ld.fecha_desbloqueo",
Principal.leerNumero(scanner));
                }
                case 0 -> { }
                default -> System.out.println("Opcion invalida.");
            }
        } while(opcion != 0);
    }

    public static void ejecutar(String sql, Object... parametros) throws SQLException
    {
        try(Connection con = BaseDeDatos.getConnection(); PreparedStatement ps = con.prepareStatement(sql))
        {
            for(int i = 0; i < parametros.length; i++)
            {
                ps.setObject(i + 1, parametros[i]);
            }
            try(ResultSet rs = ps.executeQuery())
            {
```

```
                ResultSetMetaData meta = rs.getMetaData();
                int columnas = meta.getColumnCount();
                int filas = 0;
                for(int i = 1; i <= columnas; i++)
                {
                    System.out.printf("%-23s", meta.getColumnLabel(i));
                }
                System.out.println();
                while(rs.next())
                {
                    for(int i = 1; i <= columnas; i++)
                    {
                        String valor = rs.getString(i);
                        System.out.printf("%-23.23s", valor == null ? "NULL" : valor);
                    }
                    System.out.println();
                    filas++;
                }
                System.out.println("Registros encontrados: " + filas);
            }
        }
    }
}
```

Explicación del archivo: Ofrece un menú con siete consultas parametrizadas. Usa PreparedStatement para los valores indicados por el usuario, ResultSet para recorrer registros y ResultSetMetaData para mostrar las columnas de cualquier consulta sin repetir código.

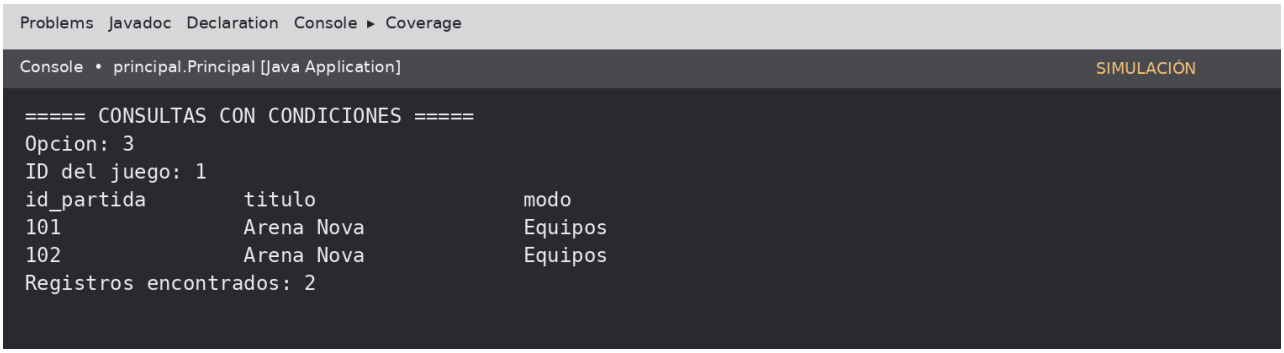


Figura 4. Simulación de una consulta SELECT con condición desde Eclipse.

2.2.4 Tres actualizaciones y tres eliminaciones

Actualización 1: usuario 10 pasa a INACTIVO. Actualización 2: juego 1 cambia a la versión 1.3. Actualización 3: el perfil del usuario 5 modifica el país. Se eliminan una relación de logro desbloqueado, una participación y una estadística por su clave compuesta. Las seis instrucciones usan WHERE, PreparedStatement y una misma conexión transaccional.

Archivo: ejemplo/src/consultas/EjerciciosSQL.java

```
package consultas;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.Scanner;

import conexion.BaseDeDatos;
import seguridad.Seguridad;

public class EjerciciosSQL {
    public static void aplicarCambios(Scanner scanner) throws SQLException
    {
        try(Connection con = BaseDeDatos.getConnection())
        {
            con.setAutoCommit(false);
            try
            {
                modificar(con, "UPDATE usuario SET estado = ? WHERE id_usuario = ?", "INACTIVO", 10);
                modificar(con, "UPDATE juego SET version_actual = ? WHERE id_juego = ?", "1.3", 1);
                modificar(con, "UPDATE perfil_jugador SET pais = ? WHERE id_usuario = ?", "Bolivia", 5);

                modificar(con, "DELETE FROM logro_desbloqueado WHERE id_usuario = ? AND id_logro = ?", 9, 9);
                modificar(con, "DELETE FROM participacion WHERE id_usuario = ? AND id_partida = ?", 9, 109);
```

```
                modificar(con, "DELETE FROM estadistica_juego WHERE id_usuario = ? AND id_juego = ?", 10, 5);

                if(Seguridad.confirmar(scanner))
                {
                    con.commit();
                    System.out.println("Las 3 actualizaciones y las 3 eliminaciones fueron confirmadas.");
                }
                else
                {
                    con.rollback();
                    System.out.println("Clave incorrecta: ninguna de las seis operaciones se guardo.");
                }
            }
            catch(SQLException e)
            {
                con.rollback();
                throw e;
            }
        }
    }

    private static void modificar(Connection con, String sql, Object... parametros) throws SQLException
    {
        try(PreparedStatement ps = con.prepareStatement(sql))
        {
            for(int i = 0; i < parametros.length; i++)
            {
                ps.setObject(i + 1, parametros[i]);
            }
            System.out.println("Registros afectados: " + ps.executeUpdate());
        }
    }
}
```

Explicación del archivo: Ejecuta tres instrucciones UPDATE y tres DELETE parametrizadas. Desactiva auto-commit antes de modificar la base, solicita una clave y confirma todo mediante commit() o revierte todo mediante rollback().

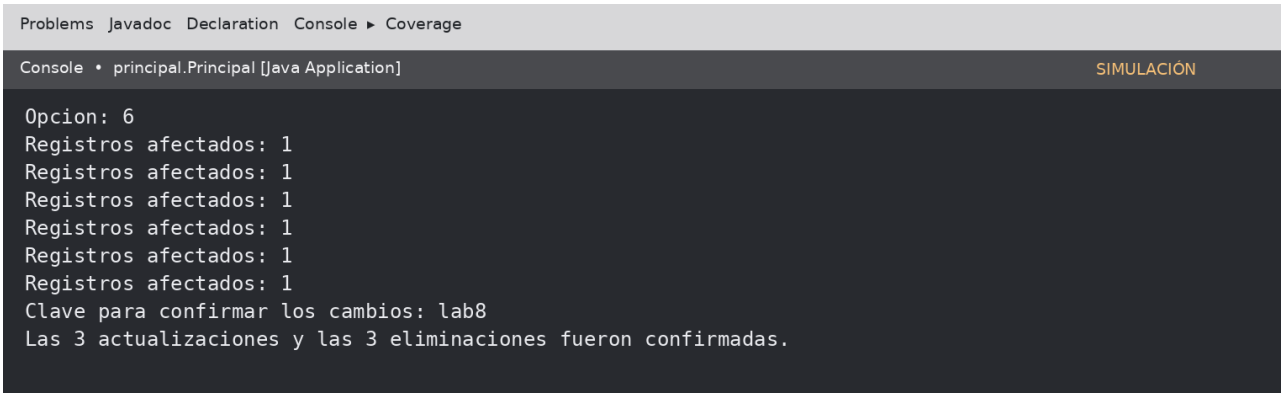


Figura 5. Simulación de tres UPDATE y tres DELETE ejecutados en una transacción.

2.3 Ejercicio 4: aplicación Java con menú CRUD

El ejercicio exige un menú para insertar, recuperar, actualizar y eliminar en al menos dos tablas si el trabajo es individual, o en todas si es grupal. El proyecto permite elegir las ocho tablas, recupera sus campos mediante PRAGMA table_info y adapta el formulario según la clave primaria, incluidas claves compuestas. Los nombres de tablas se seleccionan desde una lista cerrada para no construir consultas con identificadores arbitrarios.

Archivo: ejemplo/src/principal/Principal.java

```
package principal;

import java.sql.Connection;
import java.sql.SQLException;
import java.util.Scanner;

import conexion.BaseDeDatos;
import conexion.DatosPrueba;
import consultas.Consultas;
import consultas.EjerciciosSQL;
```

```
import gestor.GestorCRUD;
import gestor.GestorRegistros;
import seguridad.Seguridad;

public class Principal {
    public static void main(String[] args)
    {
        try(Scanner scanner = new Scanner(System.in))
        {
            BaseDeDatos.inicializar();
            System.out.println("Base de datos inicializada correctamente.");
            int opcion;

            do
            {
                System.out.println("\n===== PLATAFORMA MULTIJUGADOR =====");
                System.out.println("1. Cargar registros de prueba");
                System.out.println("2. Consultas SQL");
                System.out.println("3. Gestionar tablas (CRUD)");
                System.out.println("4. Consultar arreglo de objetos");
                System.out.println("5. Comprobar transacciones");
                System.out.println("6. Ejecutar 3 UPDATE y 3 DELETE");
                System.out.println("0. Salir");
                System.out.print("Opcion: ");
                opcion = leerNumero(scanner);

                try
                {
                    switch(opcion)
                    {
                        case 1 -> cargarDatos(scanner);
                        case 2 -> Consultas.menu(scanner);
                        case 3 -> GestorCRUD.menu(scanner);
                        case 4 -> GestorRegistros.menu(scanner);
                        case 5 -> comprobarTransaccion(scanner);
                        case 6 -> EjerciciosSQL.aplicarCambios(scanner);
                        case 0 -> System.out.println("Hasta luego.");
                        default -> System.out.println("Opcion invalida.");
                    }
                }
                catch(SQLException e)
                {
                    System.out.println("Error de base de datos: " + e.getMessage());
                }
            } while(opcion != 0);
        }
        catch(SQLException e)
        {
            System.out.println("No se pudo iniciar: " + e.getMessage());
        }
    }

    public static int leerNumero(Scanner scanner)
    {
        try
        {
            return Integer.parseInt(scanner.nextLine().trim());
        }
        catch(NumberFormatException e)
        {
            return -1;
        }
    }

    private static void cargarDatos(Scanner scanner) throws SQLException
    {
        try(Connection con = BaseDeDatos.getConnection())
        {
            con.setAutoCommit(false);
            try
            {
                DatosPrueba.cargar(con);
                if(Seguridad.confirmar(scanner))
                {
                    con.commit();
                    System.out.println("Carga confirmada: 10 registros por tabla como minimo.");
                }
                else
                {
                    con.rollback();
                    System.out.println("Clave incorrecta. Se revirtio la carga.");
                }
```

```
            }
            catch(SQLException e)
            {
                con.rollback();
                throw e;
            }
        }
    }

    private static void comprobarTransaccion(Scanner scanner) throws SQLException
    {
        try(Connection con = BaseDeDatos.getConnection())
        {
            con.setAutoCommit(false);
            try
            {
                String sql = "UPDATE usuario SET estado = estado WHERE id_usuario = 1";
                try(var ps = con.prepareStatement(sql))
                {
                    int filas = ps.executeUpdate();
                    System.out.println("Registros afectados antes de confirmar: " + filas);
                }
                if(Seguridad.confirmar(scanner))
                {
                    con.commit();
                    System.out.println("Transaccion confirmada con commit().");
                }
                else
                {
                    con.rollback();
                    System.out.println("Transaccion revertida con rollback().");
                }
            }
            catch(SQLException e)
            {
                con.rollback();
                throw e;
            }
        }
    }
}
```

Explicación del archivo: Punto de entrada de la aplicación. Inicializa la base, muestra el menú general y deriva la ejecución hacia carga de datos, consultas, CRUD, consulta sobre arreglos, verificación de transacciones y cambios filtrados. Un Scanner gestiona la interacción por consola.

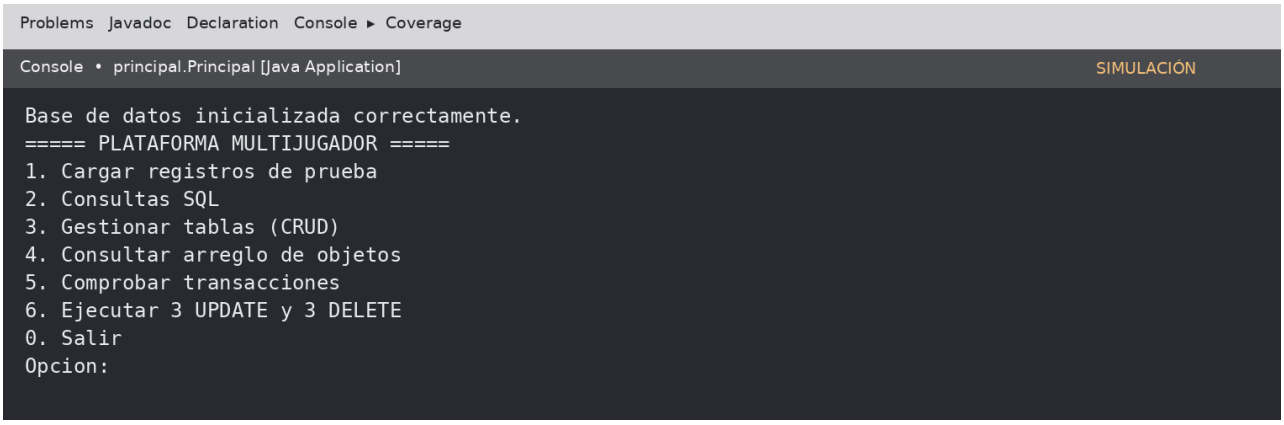


Figura 6. Simulación del menú principal de la plataforma multijugador.

Archivo: ejemplo/src/gestor/GestorCRUD.java

```
package gestor;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

import conexion.BaseDeDatos;
```

```
import principal.Principal;
import seguridad.Seguridad;

public class GestorCRUD {
    public static final String[] TABLAS = {
            "usuario", "perfil_jugador", "juego", "partida", "participacion",
            "estadistica_juego", "logro", "logro_desbloqueado"
    };

    private record Campo(String nombre, String tipo, boolean obligatorio, int clave, String defecto) { }

    public static boolean esTablaValida(String tabla)
    {
        for(String nombre : TABLAS)
        {
            if(nombre.equals(tabla))
            {
                return true;
            }
        }
        return false;
    }

    public static void menu(Scanner scanner) throws SQLException
    {
        System.out.println("\n===== TABLAS DISPONIBLES =====");
        for(int i = 0; i < TABLAS.length; i++)
        {
            System.out.println((i + 1) + ". " + TABLAS[i]);
        }
        System.out.print("Seleccione una tabla: ");
        int indice = Principal.leerNumero(scanner) - 1;
        if(indice < 0 || indice >= TABLAS.length)
        {
            System.out.println("Tabla invalida.");
            return;
        }
        String tabla = TABLAS[indice];
        int opcion;
        do
        {
            System.out.println("\n===== " + tabla.toUpperCase() + " =====");
            System.out.println("1. Insertar registro");
            System.out.println("2. Mostrar registros");
            System.out.println("3. Actualizar registro");
            System.out.println("4. Eliminar registro");
            System.out.println("0. Volver");
            System.out.print("Opcion: ");
            opcion = Principal.leerNumero(scanner);
            switch(opcion)
            {
                case 1 -> insertar(tabla, scanner);
                case 2 -> mostrar(tabla);
                case 3 -> actualizar(tabla, scanner);
                case 4 -> eliminar(tabla, scanner);
                case 0 -> { }
                default -> System.out.println("Opcion invalida.");
            }
        } while(opcion != 0);
    }

    private static List<Campo> obtenerCampos(Connection con, String tabla) throws SQLException
    {
        List<Campo> campos = new ArrayList<>();
        try(Statement stmt = con.createStatement(); ResultSet rs = stmt.executeQuery("PRAGMA table_info(" + tabla
+ ")"))
        {
            while(rs.next())
            {
                campos.add(new Campo(rs.getString("name"), rs.getString("type"), rs.getInt("notnull") != 0,
                        rs.getInt("pk"), rs.getString("dflt_value")));
            }
        }
        return campos;
    }

    private static Object pedirDato(Scanner scanner, Campo campo, boolean admiteVacio)
    {
        while(true)
        {
            System.out.print(campo.nombre() + " (" + campo.tipo() + ")" + (admiteVacio ? " [vacio = NULL o valor
predeterminado]" : "") + ": ");
```

```
            String valor = scanner.nextLine().trim();
            if(valor.isEmpty())
            {
                if(admiteVacio)
                {
                    return null;
                }
                System.out.println("El campo es obligatorio.");
                continue;
            }
            if(campo.tipo().toUpperCase().contains("INT"))
            {
                try
                {
                    return Integer.parseInt(valor);
                }
                catch(NumberFormatException e)
                {
                    System.out.println("Debe ingresar un numero entero.");
                    continue;
                }
            }
            return valor;
        }
    }

    private static void insertar(String tabla, Scanner scanner) throws SQLException
    {
        try(Connection con = BaseDeDatos.getConnection())
        {
            List<Campo> campos = obtenerCampos(con, tabla);
            List<String> nombres = new ArrayList<>();
            List<Object> valores = new ArrayList<>();
            for(Campo campo : campos)
            {
                boolean claveAutomatica = (tabla.equals("usuario") || tabla.equals("juego") ||
tabla.equals("partida") || tabla.equals("logro")) && campo.clave() == 1 &&
campo.tipo().equalsIgnoreCase("INTEGER");
                boolean opcional = campo.defecto() != null || !campo.obligatorio() && campo.clave() == 0 ||
claveAutomatica;
                Object valor = pedirDato(scanner, campo, opcional);
                if(valor != null)
                {
                    nombres.add(campo.nombre());
                    valores.add(valor);
                }
            }
            String sql;
            if(nombres.isEmpty())
            {
                sql = "INSERT INTO " + tabla + " DEFAULT VALUES";
            }
            else
            {
                sql = "INSERT INTO " + tabla + " (" + String.join(", ", nombres) + ") VALUES (" + String.join(",
", java.util.Collections.nCopies(nombres.size(), "?")) + ")";
            }
            con.setAutoCommit(false);
            if(tabla.equals("participacion") && !puedeIngresar(con, nombres, valores))
            {
                con.rollback();
                System.out.println("El usuario ya participa en otra partida activa.");
                return;
            }
            try(PreparedStatement ps = con.prepareStatement(sql))
            {
                for(int i = 0; i < valores.size(); i++)
                {
                    ps.setObject(i + 1, valores.get(i));
                }
                confirmarCambio(con, ps, scanner);
            }
            catch(SQLException e)
            {
                con.rollback();
                System.out.println("No se pudo insertar: " + e.getMessage());
            }
        }
    }

    private static void mostrar(String tabla) throws SQLException
    {
```

```
        try(Connection con = BaseDeDatos.getConnection(); Statement stmt = con.createStatement(); ResultSet rs =
stmt.executeQuery("SELECT * FROM " + tabla + " LIMIT 100"))
        {
            var meta = rs.getMetaData();
            int columnas = meta.getColumnCount();
            for(int i = 1; i <= columnas; i++)
            {
                System.out.printf("%-20.20s", meta.getColumnName(i));
            }
            System.out.println();
            int filas = 0;
            while(rs.next())
            {
                for(int i = 1; i <= columnas; i++)
                {
                    Object valor = rs.getObject(i);
                    System.out.printf("%-20.20s", valor == null ? "NULL" : valor.toString());
                }
                System.out.println();
                filas++;
            }
            System.out.println("Registros mostrados: " + filas);
        }
    }

    private static List<Campo> claves(List<Campo> campos) throws SQLException
    {
        List<Campo> claves = campos.stream().filter(c -> c.clave() > 0).sorted((a, b) ->
Integer.compare(a.clave(), b.clave())).toList();
        if(claves.isEmpty())
        {
            throw new SQLException("Esta tabla no tiene una clave primaria definida.");
        }
        return claves;
    }

    private static List<Object> pedirClaves(Scanner scanner, List<Campo> claves)
    {
        List<Object> valores = new ArrayList<>();
        System.out.println("Identifique el registro por su clave primaria:");
        for(Campo clave : claves)
        {
            valores.add(pedirDato(scanner, clave, false));
        }
        return valores;
    }

    private static String condicion(List<Campo> claves)
    {
        return String.join(" AND ", claves.stream().map(c -> c.nombre() + " = ?").toList());
    }

    private static void actualizar(String tabla, Scanner scanner) throws SQLException
    {
        try(Connection con = BaseDeDatos.getConnection())
        {
            List<Campo> campos = obtenerCampos(con, tabla);
            List<Campo> claves = claves(campos);
            System.out.println("Campos modificables:");
            for(Campo campo : campos)
            {
                if(campo.clave() == 0)
                {
                    System.out.println("- " + campo.nombre());
                }
            }
            System.out.print("Campo que desea modificar: ");
            String nombre = scanner.nextLine().trim();
            Campo seleccionado = null;
            for(Campo campo : campos)
            {
                if(campo.nombre().equals(nombre) && campo.clave() == 0)
                {
                    seleccionado = campo;
                }
            }
            if(seleccionado == null)
            {
                System.out.println("Campo invalido.");
                return;
            }
            Object nuevoValor = pedirDato(scanner, seleccionado, !seleccionado.obligatorio());
```

```
            List<Object> ids = pedirClaves(scanner, claves);
            String sql = "UPDATE " + tabla + " SET " + nombre + " = ? WHERE " + condicion(claves);
            con.setAutoCommit(false);
            try(PreparedStatement ps = con.prepareStatement(sql))
            {
                ps.setObject(1, nuevoValor);
                for(int i = 0; i < ids.size(); i++)
                {
                    ps.setObject(i + 2, ids.get(i));
                }
                confirmarCambio(con, ps, scanner);
            }
            catch(SQLException e)
            {
                con.rollback();
                System.out.println("No se pudo actualizar: " + e.getMessage());
            }
        }
    }

    private static void eliminar(String tabla, Scanner scanner) throws SQLException
    {
        try(Connection con = BaseDeDatos.getConnection())
        {
            List<Campo> claves = claves(obtenerCampos(con, tabla));
            List<Object> ids = pedirClaves(scanner, claves);
            String sql = "DELETE FROM " + tabla + " WHERE " + condicion(claves);
            con.setAutoCommit(false);
            try(PreparedStatement ps = con.prepareStatement(sql))
            {
                for(int i = 0; i < ids.size(); i++)
                {
                    ps.setObject(i + 1, ids.get(i));
                }
                confirmarCambio(con, ps, scanner);
            }
            catch(SQLException e)
            {
                con.rollback();
                System.out.println("No se pudo eliminar: " + e.getMessage());
            }
        }
    }

    private static boolean puedeIngresar(Connection con, List<String> nombres, List<Object> valores) throws
SQLException
    {
        if(!nombres.contains("id_usuario") || !nombres.contains("id_partida"))
        {
            return false;
        }
        int idUsuario = ((Number) valores.get(nombres.indexOf("id_usuario"))).intValue();
        int idPartida = ((Number) valores.get(nombres.indexOf("id_partida"))).intValue();
        try(PreparedStatement ps = con.prepareStatement("SELECT estado FROM partida WHERE id_partida = ?"))
        {
            ps.setInt(1, idPartida);
            try(ResultSet rs = ps.executeQuery())
            {
                if(!rs.next())
                {
                    return false;
                }
                if(!"EN_CURSO".equalsIgnoreCase(rs.getString(1)))
                {
                    return true;
                }
            }
        }
        String sql = "SELECT COUNT(*) FROM participacion pa JOIN partida p ON pa.id_partida = p.id_partida WHERE
pa.id_usuario = ? AND pa.id_partida <> ? AND p.estado = ?";
        try(PreparedStatement ps = con.prepareStatement(sql))
        {
            ps.setInt(1, idUsuario);
            ps.setInt(2, idPartida);
            ps.setString(3, "EN_CURSO");
            try(ResultSet rs = ps.executeQuery())
            {
                return rs.next() && rs.getInt(1) == 0;
            }
        }
    }
```

```
    private static void confirmarCambio(Connection con, PreparedStatement ps, Scanner scanner) throws SQLException
    {
        int filas = ps.executeUpdate();
        System.out.println("Registros afectados: " + filas);
        if(Seguridad.confirmar(scanner))
        {
            con.commit();
            System.out.println("Cambios confirmados con commit().");
        }
        else
        {
            con.rollback();
            System.out.println("Clave incorrecta. Cambios revertidos con rollback().");
        }
    }
}
```

Explicación del archivo: Implementa las cuatro operaciones sobre las ocho tablas. Obtiene metadatos para pedir valores y detectar claves primarias simples o compuestas, utiliza consultas preparadas para los datos y solicita una clave antes de confirmar modificaciones. La opción de insertar en participacion comprueba la existencia de otras partidas EN_CURSO del mismo usuario.

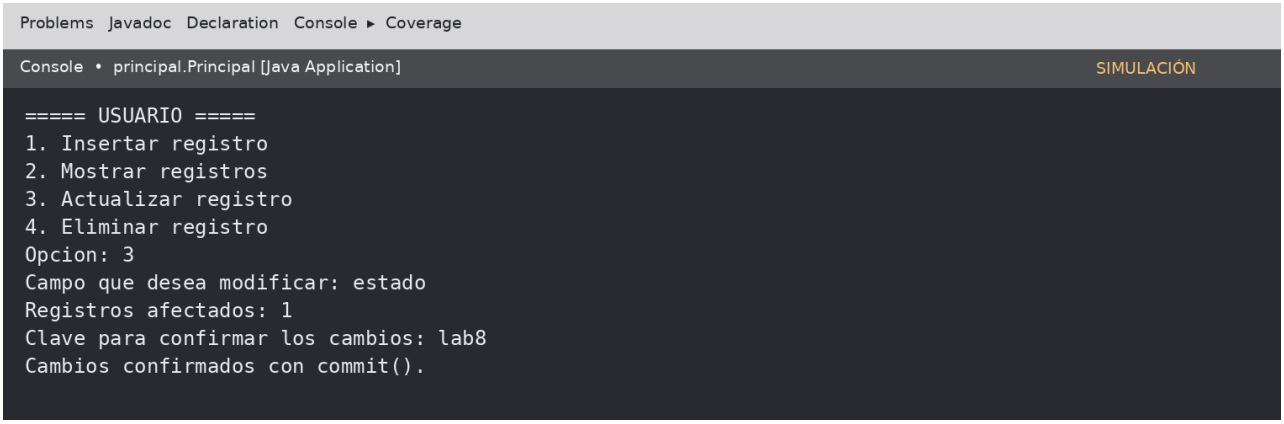


Figura 7. Simulación del menú CRUD y confirmación de cambios.

2.4 Ejercicio 5: contraseña y control de transacciones

La contraseña demostrativa se solicita después de realizar una operación dentro de la transacción. Cuando coincide con la clave configurada, se ejecuta commit(); cuando no coincide, rollback(). La clave por defecto se emplea únicamente para probar la lógica. No representa autenticación de producción, y los registros de usuarios no contienen contraseñas utilizables.

Archivo: ejemplo/src/seguridad/Seguridad.java

```
package seguridad;

import java.util.Scanner;

public class Seguridad {
    public static boolean confirmar(Scanner scanner)
    {
        String clave = System.getenv("LAB8_CLAVE");
        if(clave == null || clave.isBlank())
        {
            clave = "lab8";
        }
        System.out.print("Clave para confirmar los cambios: ");
        String ingresada = scanner.nextLine();
        return ingresada.equals(clave);
    }
}
```

Explicación del archivo: Centraliza la verificación de la clave didáctica y permite sustituir su valor mediante una variable de entorno. Devuelve true o false para que los métodos de transacción decidan entre commit() y rollback().

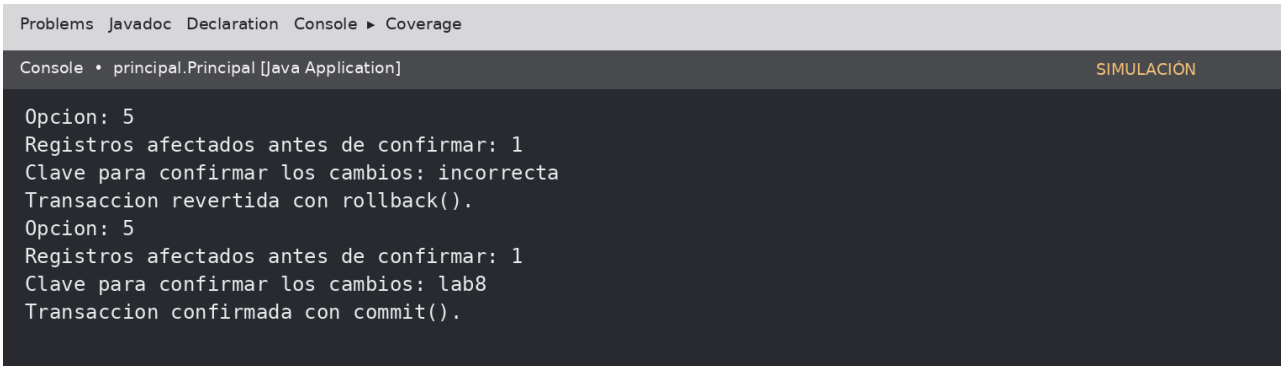


Figura 8. Simulación de comprobación de contraseña y reversión con rollback().

2.5 Ejercicio 6: clase gestora y arreglo de objetos

Se implementó una clase gestora que carga registros SQL en un arreglo de objetos y permite construir consultas desde Java. El usuario indica qué campos mostrar, una condición opcional, el orden ascendente o descendente y el límite máximo. El filtrado y el ordenamiento se realizan sobre el arreglo ya cargado, no mediante un SELECT con WHERE; esto responde a la distinción solicitada en la práctica.

Archivo: ejemplo/src/modelo/Registro.java

```
package modelo;

import java.util.LinkedHashMap;
import java.util.Map;

public class Registro {
    private final Map<String, Object> campos;

    public Registro(Map<String, Object> campos)
    {
        this.campos = new LinkedHashMap<>(campos);
    }

    public Object get(String nombre)
    {
        return campos.get(nombre);
    }

    public boolean contiene(String nombre)
    {
        return campos.containsKey(nombre);
    }

    public String[] obtenerCampos()
    {
        return campos.keySet().toArray(new String[0]);
    }
}
```

Explicación del archivo: Representa una fila como objeto Java con nombres y valores asociados. Permite consultar atributos por su nombre sin fijar el modelo a una sola tabla y obtener las columnas disponibles para las consultas dinámicas.

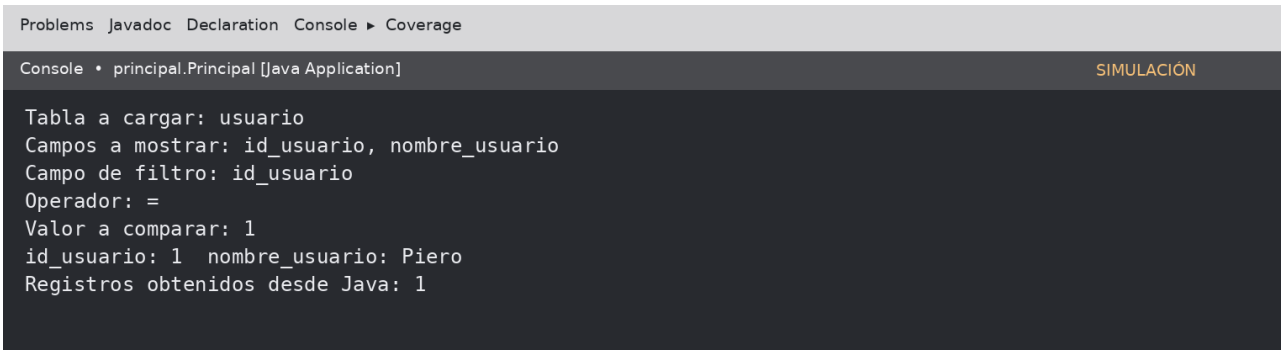


Figura 9. Simulación de los registros recuperados y convertidos en objetos Java.

Archivo: ejemplo/src/gestor/GestorRegistros.java

```
package gestor;

import java.sql.Connection;
import java.sql.ResultSet;
import java.sql.ResultSetMetaData;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Comparator;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Scanner;

import conexion.BaseDeDatos;
import modelo.Registro;
import principal.Principal;

public class GestorRegistros {
    private Registro[] registros;

    public GestorRegistros()
    {
        registros = new Registro[0];
    }

    public void cargar(String tabla) throws SQLException
    {
        if(!GestorCRUD.esTablaValida(tabla))
        {
            throw new SQLException("Tabla no permitida.");
        }
        List<Registro> lista = new ArrayList<>();
        try(Connection con = BaseDeDatos.getConnection(); Statement stmt = con.createStatement(); ResultSet rs =
stmt.executeQuery("SELECT * FROM " + tabla))
        {
            ResultSetMetaData meta = rs.getMetaData();
            while(rs.next())
            {
                Map<String, Object> campos = new LinkedHashMap<>();
                for(int i = 1; i <= meta.getColumnCount(); i++)
                {
                    campos.put(meta.getColumnName(i), rs.getObject(i));
                }
                lista.add(new Registro(campos));
            }
        }
        registros = lista.toArray(new Registro[0]);
    }

    public Registro[] consultar(String[] campos, String campoFiltro, String operador, String valor, String
campoOrden, boolean ascendente, int limite)
    {
        List<Registro> encontrados = new ArrayList<>();
        for(Registro registro : registros)
        {
            if(campoFiltro.isBlank() || cumple(registro.get(campoFiltro), operador, valor))
            {
                encontrados.add(registro);
            }
        }
        if(!campoOrden.isBlank())
        {
            Comparator<Registro> comparador = (a, b) -> comparar(a.get(campoOrden), b.get(campoOrden));
            encontrados.sort(ascendente ? comparador : comparador.reversed());
        }
        if(limite > 0 && encontrados.size() > limite)
        {
            encontrados = new ArrayList<>(encontrados.subList(0, limite));
        }
        for(Registro registro : encontrados)
        {
            for(String campo : campos)
            {
                System.out.print(campo + ": " + registro.get(campo) + "  ");
            }
            System.out.println();
        }
        System.out.println("Registros obtenidos desde Java: " + encontrados.size());
```

```
        return encontrados.toArray(new Registro[0]);
    }

    private boolean cumple(Object actual, String operador, String valor)
    {
        if(actual == null)
        {
            return operador.equals("=") && valor.equalsIgnoreCase("NULL");
        }
        int comparacion = comparar(actual, valor);
        return switch(operador)
        {
            case "=" -> comparacion == 0;
            case "!=" -> comparacion != 0;
            case ">" -> comparacion > 0;
            case ">=" -> comparacion >= 0;
            case "<" -> comparacion < 0;
            case "<=" -> comparacion <= 0;
            case "contiene" -> actual.toString().toLowerCase().contains(valor.toLowerCase());
            default -> false;
        };
    }

    private int comparar(Object a, Object b)
    {
        if(a == null && b == null) return 0;
        if(a == null) return -1;
        if(b == null) return 1;
        try
        {
            return Double.compare(Double.parseDouble(a.toString()), Double.parseDouble(b.toString()));
        }
        catch(NumberFormatException e)
        {
            return a.toString().compareToIgnoreCase(b.toString());
        }
    }

    public static void menu(Scanner scanner) throws SQLException
    {
        System.out.println("\nTablas disponibles: " + String.join(", ", GestorCRUD.TABLAS));
        System.out.print("Tabla a cargar: ");
        String tabla = scanner.nextLine().trim().toLowerCase();
        if(!GestorCRUD.esTablaValida(tabla))
        {
            System.out.println("Tabla invalida.");
            return;
        }
        GestorRegistros gestor = new GestorRegistros();
        gestor.cargar(tabla);
        if(gestor.registros.length == 0)
        {
            System.out.println("No hay registros en esa tabla.");
            return;
        }
        String[] disponibles = gestor.registros[0].obtenerCampos();
        System.out.println("Campos: " + String.join(", ", disponibles));
        System.out.print("Campos a mostrar (separados por coma o *): ");
        String entrada = scanner.nextLine().trim();
        String[] campos = entrada.equals("*") ? disponibles :
Arrays.stream(entrada.split(",")).map(String::trim).toArray(String[]::new);
        for(String campo : campos)
        {
            if(!gestor.registros[0].contiene(campo))
            {
                System.out.println("Campo no valido: " + campo);
                return;
            }
        }
        System.out.print("Campo de filtro (vacio = ninguno): ");
        String campoFiltro = scanner.nextLine().trim();
        if(!campoFiltro.isEmpty() && !gestor.registros[0].contiene(campoFiltro))
        {
            System.out.println("Campo de filtro invalido.");
            return;
        }
        String operador = "=";
        String valor = "";
        if(!campoFiltro.isEmpty())
        {
            System.out.print("Operador (=, !=, >, >=, <, <=, contiene): ");
            operador = scanner.nextLine().trim();
```

```
            if(!Arrays.asList("=", "!=", ">", ">=", "<", "<=", "contiene").contains(operador))
            {
                System.out.println("Operador invalido.");
                return;
            }
            System.out.print("Valor a comparar: ");
            valor = scanner.nextLine();
        }
        System.out.print("Campo de orden (vacio = ninguno): ");
        String campoOrden = scanner.nextLine().trim();
        if(!campoOrden.isEmpty() && !gestor.registros[0].contiene(campoOrden))
        {
            System.out.println("Campo de orden invalido.");
            return;
        }
        System.out.print("Orden (ASC/DESC): ");
        boolean ascendente = !scanner.nextLine().trim().equalsIgnoreCase("DESC");
        System.out.print("Limite de registros (0 = todos): ");
        int limite = Principal.leerNumero(scanner);
        gestor.consultar(campos, campoFiltro, operador, valor, campoOrden, ascendente, limite);
    }
}
```

Explicación del archivo: Carga filas de cualquier tabla autorizada en Registro[]. El método consultar permite escoger columnas, aplicar filtros con operadores, ordenar objetos mediante Comparator y limitar los resultados. La selección y el procesamiento de las filas se ejecutan en memoria.

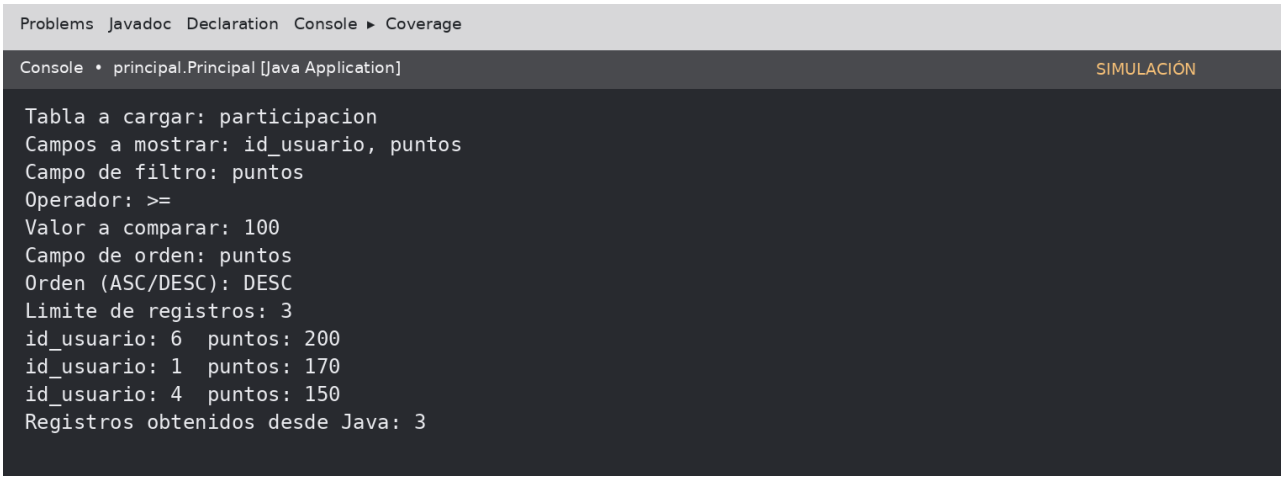


Figura 10. Simulación de filtros, orden y límite aplicados sobre Registro[].

2.6 Archivo de configuración del módulo Java

Se mantiene module-info.java tal como existe en el proyecto Eclipse. La sentencia requires java.sql habilita el acceso a JDBC desde el módulo ejemplo. La biblioteca sqlite-jdbc sigue declarada como dependencia externa del proyecto original y deberá estar correctamente ubicada en Eclipse.

Archivo: ejemplo/src/module-info.java

```
/**
 *
 */
/**
 *
 */
module ejemplo {
    requires java.sql;
}
```

Explicación del archivo: Declara el módulo Java del proyecto y habilita el acceso al módulo java.sql. No crea conexiones ni tablas: permite compilar las clases JDBC del proyecto modular.

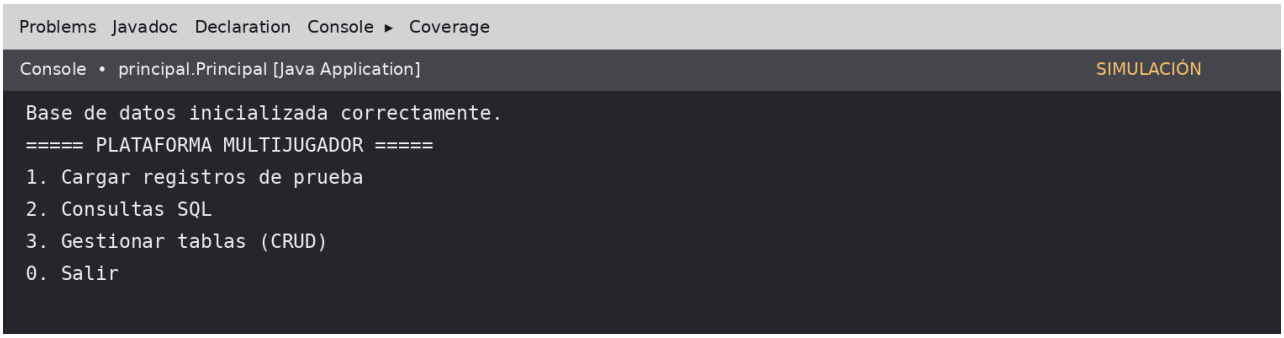


Figura 11. Simulación del arranque del proyecto Java desde Eclipse.

2.7 Comprobación adicional del esquema y las operaciones

El repositorio incluye un script de verificación SQL independiente del controlador JDBC. Para distinguir los resultados realmente comprobados de la simulación visual, se registraron ocho tablas, diez filas iniciales en cada una, siete consultas, un intento de duplicación 1:1 rechazado, una FK inexistente rechazada, un rollback correcto y seis modificaciones filtradas. La ejecución gráfica de Eclipse no se presenta como prueba real.

Archivo: docs/verificar_sqlite.py

```
import ast
import json
import re
import sqlite3
from pathlib import Path

raiz = Path(__file__).resolve().parents[1]
base = (raiz / 'ejemplo/src/conexion/BaseDeDatos.java').read_text()
datos = (raiz / 'ejemplo/src/conexion/DatosPrueba.java').read_text()
sqls = re.findall(r'String sql\w+ = """(.*?)""";', base, re.S)
con = sqlite3.connect(':memory:')
con.execute('PRAGMA foreign_keys=ON')
for sql in sqls:
    con.execute(sql)

bloques = re.findall(r'insertar\(con, "([^"]+)", new Object\[\]\[\] \{(.*?)\}\);', datos, re.S)
assert len(bloques) == 8, len(bloques)
for sql, bloque in bloques:
    registros = []
    for fila in re.findall(r'\{([^{}]+)\}', bloque):
        valores = re.findall(r'"(?:[^"\\]|\\.)*"|null|-?\d+', fila)
        registros.append([None if v == 'null' else json.loads(v) if v.startswith('"') else int(v) for v in
valores])
    assert len(registros) >= 10, (sql, len(registros))
    con.executemany(sql, registros)
con.commit()

conteos = con.execute("SELECT name FROM sqlite_master WHERE type='table' ORDER BY name").fetchall()
print('TABLAS CREADAS:', len(conteos))
for (tabla,) in conteos:
    total = con.execute(f'SELECT COUNT(*) FROM {tabla}').fetchone()[0]
    print(f'{tabla:<22}: {total} registros')
    assert total >= 10, (tabla, total)

consultas = [
    ('usuarios_activos', "SELECT id_usuario, nombre_usuario, estado FROM usuario WHERE estado = ? ORDER BY
nombre_usuario", ('ACTIVO',)),
    ('juegos_capacidad', "SELECT id_juego, titulo, max_jugadores FROM juego WHERE max_jugadores >= ? ORDER BY
max_jugadores DESC", (4,)),
    ('partidas_por_juego', "SELECT p.id_partida, j.titulo, p.modo, p.estado FROM partida p JOIN juego j ON
p.id_juego = j.id_juego WHERE j.id_juego = ? ORDER BY p.id_partida", (1,)),
    ('historial_usuario', "SELECT u.nombre_usuario, j.titulo, p.id_partida, pa.puntos, pa.resultado FROM
participacion pa JOIN usuario u ON pa.id_usuario = u.id_usuario JOIN partida p ON pa.id_partida = p.id_partida
JOIN juego j ON p.id_juego = j.id_juego WHERE u.id_usuario = ? ORDER BY p.inicio DESC", (1,)),
    ('puntaje_minimo', "SELECT u.nombre_usuario, pa.id_partida, pa.puntos, pa.bajas FROM participacion pa JOIN
usuario u ON pa.id_usuario = u.id_usuario WHERE pa.puntos >= ? ORDER BY pa.puntos DESC", (100,)),
    ('finalizadas_region', "SELECT p.id_partida, j.titulo, p.region, p.fin FROM partida p JOIN juego j ON
p.id_juego = j.id_juego WHERE p.estado = ? AND p.region = ? ORDER BY p.fin DESC", ('FINALIZADA','LATAM')),
    ('logros_usuario', "SELECT u.nombre_usuario, l.titulo, j.titulo AS juego, ld.fecha_desbloqueo FROM
logro_desbloqueado ld JOIN usuario u ON ld.id_usuario = u.id_usuario JOIN logro l ON ld.id_logro = l.id_logro JOIN
juego j ON l.id_juego = j.id_juego WHERE u.id_usuario = ? ORDER BY ld.fecha_desbloqueo", (1,)),
]
for nombre, sql, params in consultas:
```

```
    filas = con.execute(sql,params).fetchall()
    assert filas, nombre
    print(f'CONSULTA {nombre:<20}: {len(filas)} filas')

try:
    con.execute("INSERT INTO perfil_jugador (id_usuario) VALUES (1)")
    raise AssertionError('Permitio duplicar perfil 1:1')
except sqlite3.IntegrityError:
    print('PRUEBA 1:1: duplicado rechazado')

try:
    con.execute("INSERT INTO participacion (id_usuario,id_partida) VALUES (999,101)")
    raise AssertionError('Permitio FK inexistente')
except sqlite3.IntegrityError:
    print('PRUEBA FK: usuario inexistente rechazado')

con.rollback()
con.execute('BEGIN')
con.execute("UPDATE juego SET version_actual=? WHERE id_juego=?",('9.9',1))
assert con.execute('SELECT version_actual FROM juego WHERE id_juego=1').fetchone()[0]=='9.9'
con.rollback()
assert con.execute('SELECT version_actual FROM juego WHERE id_juego=1').fetchone()[0]=='1.2'
print('PRUEBA ROLLBACK: cambio revertido correctamente')

con.execute('BEGIN')
cambios = [
    ("UPDATE usuario SET estado = ? WHERE id_usuario = ?", ('INACTIVO',10)),
    ("UPDATE juego SET version_actual = ? WHERE id_juego = ?", ('1.3',1)),
    ("UPDATE perfil_jugador SET pais = ? WHERE id_usuario = ?", ('Bolivia',5)),
    ("DELETE FROM logro_desbloqueado WHERE id_usuario = ? AND id_logro = ?", (9,9)),
    ("DELETE FROM participacion WHERE id_usuario = ? AND id_partida = ?", (9,109)),
    ("DELETE FROM estadistica_juego WHERE id_usuario = ? AND id_juego = ?", (10,5)),
]
for sql, args in cambios:
    cur=con.execute(sql,args)
    assert cur.rowcount == 1,(sql,cur.rowcount)
con.commit()
print('PRUEBA 3 UPDATE + 3 DELETE: seis operaciones confirmadas')
assert not con.execute('PRAGMA foreign_key_check').fetchall()
print('PRUEBA FINAL FK: sin referencias invalidas')
print('RESULTADO: VALIDACION SQL SUPERADA')
```

Explicación del archivo: Script adicional de prueba que utiliza sqlite3 de Python como verificador independiente del esquema y las sentencias SQL. Comprueba los requisitos básicos de cardinalidad, consultas y transacciones antes de la validación final en Eclipse.

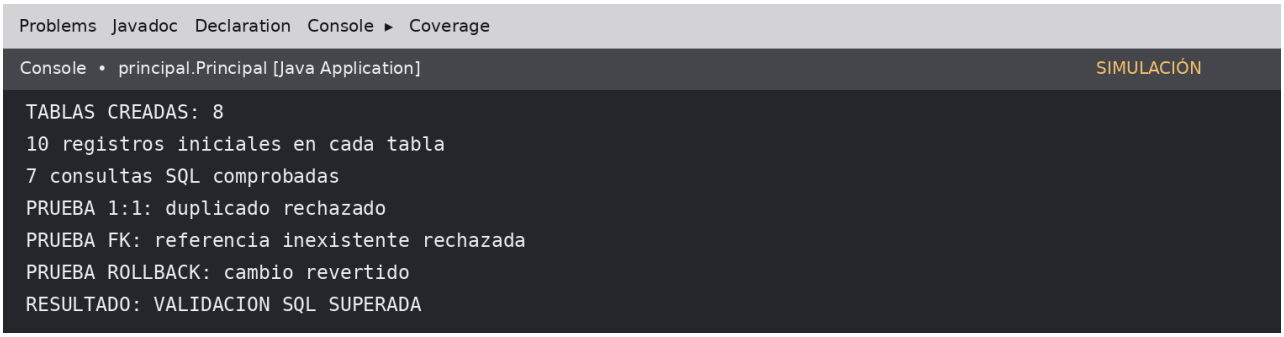


Figura 12. Simulación visual del resumen de comprobaciones del script SQL.

3. CUESTIONARIO

1. ¿Qué es SQLite y cuáles son sus principales características que lo diferencian de otros sistemas de bases de datos?

SQLite es un sistema gestor de bases de datos relacional embebido, transaccional y sin servidor independiente: almacena normalmente toda la base en un archivo. Es ligero y fácil de distribuir, aunque aplicaciones con muchas escrituras concurrentes suelen necesitar una arquitectura diferente. Es adecuado para este laboratorio porque JDBC puede abrir directamente multijugador.db (SQLite, s. f.-c).

2. ¿Qué es JDBC y cómo se utiliza para conectarse a una base de datos en Java?

JDBC es la API estándar de Java para interactuar con bases de datos mediante interfaces como Connection, Statement, PreparedStatement y ResultSet. El controlador sqlite-jdbc traduce esas operaciones al motor SQLite; al abrir una URL jdbc:sqlite:... se obtiene una conexión capaz de ejecutar SQL (Oracle, s. f.-a; Xerial, s. f.).

3. Explica el propósito de la clase DriverManager en JDBC.

DriverManager administra controladores JDBC disponibles y proporciona conexiones para una URL determinada. En BaseDeDatos.getConnection(), el método getConnection(URL) recupera una conexión al archivo SQLite; si el driver externo no está presente, la apertura fallará (Oracle, s. f.-a).

4. ¿Cómo se utiliza para obtener una conexión a una base de datos y qué parámetros son necesarios para conectarse a SQLite?

Se llama a DriverManager.getConnection() con una URL de formato jdbc:sqlite:ruta_del_archivo.db. Para una base local de SQLite normalmente no se requieren usuario ni contraseña de un servidor. En esta práctica se usa jdbc:sqlite:multijugador.db, y después se activa PRAGMA foreign_keys = ON en esa conexión (Xerial, s. f.; SQLite, s. f.-a).

5. ¿Qué es una sentencia SQL y cómo puedes ejecutarla en SQLite usando JDBC?

Una sentencia SQL es una instrucción para crear estructuras o consultar y modificar datos, por ejemplo CREATE TABLE, SELECT o UPDATE. Desde JDBC se prepara un Statement o PreparedStatement y se invoca execute(), executeUpdate() o executeQuery() según el tipo de instrucción y el resultado esperado (Oracle, s. f.-b).

6. Explica la diferencia entre Statement y PreparedStatement en JDBC.

Statement ejecuta texto SQL que se suministra directamente al método. PreparedStatement representa una sentencia preparada con marcadores ?, cuyos valores se envían por métodos setString(), setInt() o setObject(). El segundo facilita reutilizar la consulta y separar instrucciones de valores (Oracle, s. f.-b).

7. ¿Cuándo es recomendable usar PreparedStatement y qué ventajas tiene en términos de seguridad?

Se recomienda cuando la consulta depende de datos ingresados por el usuario, en especial búsquedas, inserciones, actualizaciones y eliminaciones. Los valores parametrizados no se interpretan como fragmentos del SQL, por lo que se reduce el riesgo de inyección SQL. Para nombres de columnas o tablas se debe validar contra listas permitidas, porque esos identificadores no se sustituyen con ? (Oracle, s. f.-b).

8. Describe los pasos para ejecutar una consulta SELECT en una tabla de SQLite y procesar los resultados usando JDBC.

Se obtiene Connection, se prepara un SELECT, se asignan sus parámetros y se llama a executeQuery(). El ResultSet resultante se recorre con next(); cada columna puede leerse mediante getString(), getInt() o getObject(). Se cierran ResultSet, PreparedStatement y Connection mediante try-with-resources, tal como ocurre en Consultas.java (Oracle, s. f.-b).

9. ¿Cómo puedes gestionar las transacciones en SQLite usando JDBC?

Se establece setAutoCommit(false), se ejecutan una o más operaciones y, si todas son correctas y el usuario autoriza el cambio, se llama a commit(). Ante un error o una clave incorrecta, rollback() revierte las modificaciones pendientes de la transacción. Conviene usar una sola conexión para las operaciones que deben confirmarse como una unidad (Oracle, s. f.-a; SQLite, s. f.-d).

10. ¿Qué es la función executeUpdate() en JDBC y cuándo se utiliza?

executeUpdate() ejecuta INSERT, UPDATE y DELETE y retorna el número de filas afectadas; también admite sentencias DDL que no devuelven ResultSet, en cuyo caso retorna cero. Para las consultas SELECT se utiliza executeQuery(). En el proyecto se usa executeUpdate() para cargar y modificar registros (Oracle, s. f.-b).

11. Describe cómo puedes manejar errores y excepciones relacionadas con la base de datos cuando trabajas con JDBC.

Las operaciones JDBC pueden lanzar SQLException. Se encapsulan en try/catch para informar el error, y try-with-resources libera conexiones y sentencias. Dentro de una transacción, al detectar una SQLException se solicita rollback() antes de propagar o mostrar el problema. Las restricciones de FOREIGN KEY o UNIQUE también se comunican mediante SQLException (Oracle, s. f.-a).

12. ¿Cómo puedes realizar consultas parametrizadas usando PreparedStatement en JDBC para mejorar el rendimiento y la seguridad?

Se prepara una instrucción SQL con parámetros ?, se asigna un valor a cada posición comenzando por 1 y se ejecuta la operación correspondiente. La sentencia puede reutilizarse variando sus valores; además, la parametrización reduce la posibilidad de que los datos de entrada alteren la estructura del SQL. En DatosPrueba.java un único PreparedStatement procesa varias filas de la misma entidad (Oracle, s. f.-b).

13. Explica el proceso de cierre de recursos (como Connection, Statement y ResultSet) en JDBC y ¿por qué es importante cerrar estos recursos correctamente?

Connection, Statement y ResultSet manejan recursos asociados a la base. Con try-with-resources se cierran automáticamente incluso cuando ocurre una excepción, lo que evita conexiones y cursores abiertos. Se recomienda crear los recursos dentro del encabezado try en el orden en que se necesitan; Java los cierra en orden inverso (Oracle, s. f.-a).

4. BIBLIOGRAFÍA

Universidad Católica de Santa María. (2026). Guía de prácticas de Lenguajes de Programación III: Sesión N.° 08, bases de datos SQLite y JDBC en Java [Material de curso]. Escuela Profesional de Ingeniería de Sistemas.

SQLite. (s. f.-a). SQLite foreign key support. https://www.sqlite.org/foreignkeys.html

SQLite. (s. f.-b). CREATE TABLE. https://www.sqlite.org/lang_createtable.html

SQLite. (s. f.-c). About SQLite. https://www.sqlite.org/about.html

SQLite. (s. f.-d). Transaction. https://www.sqlite.org/lang_transaction.html

Oracle. (s. f.-a). Connection (Java SE 21 & JDK 21). https://docs.oracle.com/en/java/javase/21/docs/api/java.sql/java/sql/Connection.html

Oracle. (s. f.-b). PreparedStatement (Java SE 21 & JDK 21). https://docs.oracle.com/en/java/javase/21/docs/api/java.sql/java/sql/PreparedStatement.html

Xerial. (s. f.). sqlite-jdbc: SQLite JDBC driver [Repositorio de código]. GitHub. https://github.com/xerial/sqlite-jdbc